

Guión

Grupo 7
Brigada 6
Práctica 3
Duración 4:43

1) Desarrollo

[Ejercicio 1: preguntar a LLM sobre el funcionamiento de dos estructuras: HashMap, TreeSet]

[Explicación de HashMap]

DIALOGO1: HashMap es una estructura que almacena datos en pares de clave-valor y que su objetivo principal es encontrar un valor específico casi al instante, sin importar cuántos millones de registros haya guardados, funciona de la siguiente forma:

DIÁLOGO2: Al ingresar una clave, como por ejemplo el nombre "*Juan*", el HashMap la pasa por una función hash matemática.

DIALOGO3: La función transforma esa clave en un índice numérico entero.

DIALOGO4: Dicho índice define la cubeta de memoria exacta (*bucket*) donde se guardará el valor asociado a "*Juan*".

DIÁLOGO5: Al buscar a "*Juan*", se repite la fórmula para ir directo a su cubeta sin recorrer las demás, quedando los datos distribuidos por cálculo matemático y no por un orden lógico.

[Mención de ejemplos]

Ejemplos comunes de aplicación

DIÁLOGO6: Sistemas de caché: Almacenan respuestas a operaciones pesadas para devolverlas al instante si se repite la misma consulta.

DIALOGO7: Conteo de frecuencias: Registran repeticiones asociando un elemento como clave y su número de apariciones como valor.

DIALOGO8: Tablas de búsqueda o diccionarios: Permiten consultar información rápidamente usando un identificador único, como los datos de un usuario mediante su ID.

[Explicación de TreeSet]

DIALOGO9: Treeset es una estructura que pertenece a la familia de los Sets. Esto significa que no permite elementos duplicados y en el caso específico del TreeSet, siempre mantiene sus elementos ordenados, funciona de la siguiente forma:

DIALOGO10: Primero, Al insertar un número por ejemplo 31, el árbol lo compara contra el "nodo raíz" ejemplo el 70.

DIALOGO11: Si el número es menor lo envía a la rama izquierda; si es mayor, a la rama derecha.

DIÁLOGO12: La estructura se reacomoda continuamente para no quedar asimétrica, permitiendo que al recorrerlo de izquierda a derecha los datos salgan perfectamente ordenados.

[Mención de ejemplos]

Ejemplos comunes de aplicación

DIALOGO13: Leaderboards (Tablas de clasificación): Gestionan puntuaciones altas en juegos sin valores repetidos por usuario, ordenándolas automáticamente de mayor a menor.

DIALOGO14: Filtros de datos únicos y ordenados: Procesan grandes volúmenes de información (como listas de correos), descartando duplicados y dejando los registros limpios y ordenados alfabéticamente.

DIALOGO15: Agendas o sistemas de reservas: Almacenan fechas u horarios ordenados cronológicamente, permitiendo consultar con rapidez cuál es el evento más próximo después de una fecha dada.

[Ejercicio 2: Verificar si es un palindromo]

DIALOGO16: En el ejercicio 2, la instrucción fue Diseñar una aplicación en Java que lea un número entero de 5 dígitos y verifique si es palíndromo y de manera obligatoria usar al menos un método estático. Como restricciones, el programa no debe aceptar números negativos ni con longitud distinta a 5 dígitos (> 9999 y < 100000); de ser así, el sistema muestra un error y solicita un nuevo valor.

DIÁLOGO17: El programa funciona así:

DIALOGO18: Mediante un bucle while El programa retiene al usuario hasta que proporcione un entero válido dentro del rango de 5 dígitos.

DIALOGO19: Se define el método evaluarPalindromo como estático. El método devuelve un valor true o false.

DIALOGO20: En lugar de buscar en posiciones de memoria usamos aritmética para recortar los pedazos del número que nos interesaban.

DIALOGO21: Primer dígito: Al hacer una división entera entre 10,000 se eliminan los últimos cuatro dígitos.

DIALOGO22: Segundo dígito: Primero divide entre 1,000 para quedarse con los dos primeros dígitos luego le aplica módulo 10 a ese resultado para atrapar el último número.

DIALOGO23: Cuarto dígito: Divide entre 10 para borrar el último dígito después, saca el módulo 10 para quedarse con el nuevo último dígito que era el cuarto originalmente.

DIALOGO24: Quinto dígito: El módulo 10 siempre devuelve el residuo de la división, el residuo siempre es el último número.

DIALOGO25: El algoritmo ignora el tercer dígito ya que al ser el centro de un número impar de cifras siempre será igual a sí mismo.

DIALOGO26: Se verifica que los extremos exteriores sean iguales ($d1 == d5$) y que los interiores coincidan ($d2 == d4$). Si ambas condiciones se cumplen, devuelve True de lo contrario devuelve False.

Finalmente, el programa muestra en pantalla si el número ingresado es o no un palíndromo.

2) Conclusión

DIÁLOGO27: Dominar HashMap y TreeSet nos permite optimizar búsquedas y mantener datos ordenados de forma automática. Al combinar esto con una sólida lógica matemática, logramos un código eficiente y limpio.